

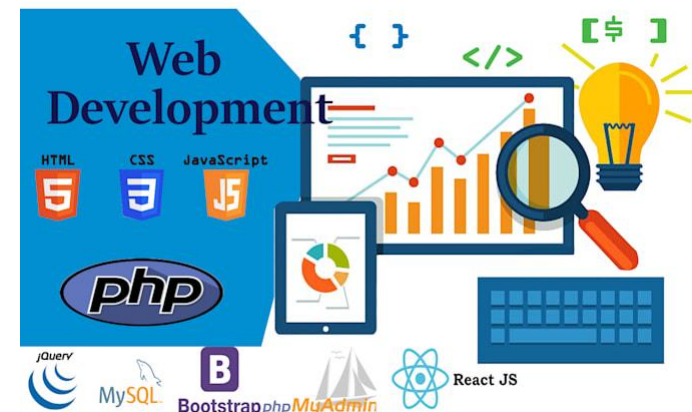
Semestre: 2026.1

Disciplina:

**ARA0062 DESENV. WEB EM HTML5, CSS,
JAVASCRIPT E PHP**

Aula 13 - SQL

Fortaleza, CE: 16 de março, 2026





Introdução ao MySQL

MySQL é um Sistema Gerenciador de Banco de Dados (SGBD) relacional.

Utiliza a linguagem SQL (Structured Query Language, ou traduzindo, Linguagem de Consulta Estruturada).

MySQL é multiusuário e multitarefas.

Introdução ao MySQL

Inicialmente desenvolvido para trabalhar com projetos de pequeno e médio porte, com a capacidade de suportar por volta de cem milhões de registros em cada tabela, podendo chegar ao tamanho médio de aproximadamente cem megabytes por tabela.

Nas versões mais recentes o MySQL ultrapassa (e muito) esses limites e capacidades das primeiras versões.

Introdução ao MySQL

MySQL é conhecido por ser de fácil utilização, e usado por empresas que trabalham com grandes volumes de dados, tais como, Google, NASA, Bradesco, HP, Sony entre outras grandes empresas de renome.

Possui uma interface extremamente simples e é compatível com grande parte dos sistemas operacionais.

Podemos dizer que essas são duas das grandes características que fazem o MySQL ser tão utilizado atualmente e estar em constante crescimento.

Compatibilidade

MySQL é compatível com a maioria dos sistemas operacionais existentes atualmente no mercado.

Pelo fato de ser desenvolvido em C e C++, isso faz com que seja extremamente fácil sua acessibilidade entre os diversos sistemas operacionais. Vejamos alguns desses sistemas.

- Windows: Compatível com todas as versões
- Linux: Sendo compatível com as principais versões, como Fedora, Core, Debian, SuSE e RedHat.
- Unix: Sendo compatível com as versões Solaris, HP-UX, AIX, SCO.
- FreeBSD.
- Mac OS X Server.

Licença

O MySQL é de Código Aberto (Open-Source), desenvolvido e distribuído sob as licenças GNU/GLP (General Public Licence, ou traduzindo, Licença Publica Geral), a qual determina o que se pode ou não fazer à ferramenta e demais recursos. Além do programa, o seu código-fonte também é disponibilizado para que qualquer usuário possa editá-lo de forma que atenda suas necessidades.

Os princípios básicos da licença GNU/GLP são:

- **Utilização:** Permite que o usuário faça uso do software para qualquer finalidade.
- **Distribuição:** Livre distribuição do software entre quaisquer pessoas.
- **Didática:** Permite que seu funcionamento seja estudado através de seu código-fonte
- **Colaboração:** Possibilita que seu código-fonte seja modificado para evoluir a ferramenta. Como regra seu novo código-fonte tem que permanecer sendo livre segundo essa licença.

Portabilidade

Devido o MySQL ter sido desenvolvido em C e C++, tornou-se extremamente fácil a portabilidade entre os diferentes sistemas, plataformas e compiladores. Possui também módulos de interface para múltiplas linguagens, tais como Delphi, Java, Python, PHP, ASP, Ruby e entre outras linguagens mais.

Formas de Armazenamento

O MySQL possibilita diversos tipos de tabela para o armazenamento dos dados, tendo em conta que cada tipo tem suas próprias características. Dessa maneira temos a possibilidade de escolhermos o tipo de acordo com cada situação diferente. Enquanto um tipo tem como prioridade a velocidade, outro da prioridade ao volume de dados, entre outras características.

Características

Velocidade

Alta velocidade no acesso dos dados em razão de diversos motivos em seu desenvolvimento com tabelas ISAM, que foi substituído pelo novo sistema MyISAM na versão 5 do MySQL, além de utilização de caches em consultas, utilização de indexação, algoritmos de busca, entre outros recursos.

Capacidade

O MySQL possui um alto poder de execução e de armazenamento. De acordo com a plataforma em que seja usado, suas tabelas poderão armazenar grandes volumes de dados, o limite ficará por conta somente do tamanho máximo de arquivos que a plataforma que estiver sendo utilizada puder manipular. Já no caso de tabelas do tipo InnoDB, onde o armazenamento pode ser realizado em um ou vários arquivos separados, fica possível armazenar volumes de dados equivalentes a TB (Terabytes) de tamanho. E referente a expressões SQL, o MySQL suporta execuções de script SQL com até 61 milhões de tabelas "joins".

Características

Capacidade (continuação)

O MySQL, por ser um banco de dados poderoso, tem a capacidade de realizar bilhões de consultas em um único dia em um site e também fazer o processamento de milhões de transações por minuto.

SQL

Como já sabemos, o MySQL trabalha com a linguagem SQL, sendo extremamente rápido. E isso foi possível devido a SQL ter sido implementada no MySQL através de códigos e funções altamente customizadas pelos seus desenvolvedores. Isso gerou a grande vantagem de velocidade no processamento dos códigos SQL. Porém, ao mesmo tempo trouxe um ponto negativo, sendo ele o fato de que com essa customização, nem todos os padrões das versões mais atuais do SQL tenham sido trazidos para o MySQL, porque poderiam prejudicar consideravelmente a velocidade do banco de dados. Entretanto, essa desvantagem influencia muito pouco na aplicação desenvolvida.

Conceito de Banco de Dados

Uma base de dados, nada mais é do que estruturas complexas de dados. Estes dados são gravados em forma de registros em tabelas.

Imagine um arquivo de fichas, numa empresa onde há várias caixas, cada uma contendo os dados dos funcionários de um certo setor.

Cada caixa possui várias fichas, que são os cadastros dos funcionários – cada ficha contém os dados de apenas um funcionário.

Indo mais longe, podemos concluir que cada ficha contém diversas informação sobre o funcionário em questão.

Portanto, cada caixa é uma tabela, contendo diversas fichas, que são os registros, e cada ficha possui várias informações sobre o funcionário, que são os campos.

Como foi dito, há várias caixas, uma para cada departamento, a soma de todas as caixas forma a base de dados.

Conceito de Banco de Dados

Observando tudo isto de fora, podemos formar o seguinte esquema:

Cenário exemplo de fichas em caixas:

Base de dados < Tabela < Registro < Coluna

Em banco de dados relacional:

Banco de dados < Tabela < Linha < Campo

Conceito de Banco de Dados

Banco de dados < Tabela < Linha < Campo

A linha acima mostra os termos normalmente usados para o que acabamos de aprender.

Os campos podem ser de diferentes tipos e tamanhos, permitindo ao programador criar tabelas que satisfaçam ao escopo do projeto.

A decisão de quais campos usar e quais não usar é muito importante, pois influi drasticamente na performance da base de dados que estamos desenvolvendo, portanto, é de bom grado um conhecimento sólido destes conceitos.

A etapa de montagem das tabelas, é senão a mais importante, uma das mais importantes etapas da montagem de uma base de dados, pois um bom projeto pode facilitar muito o trabalho de programação.

Campos de Banco de Dados

Campos

Como já sabemos, os campos são a parte fundamental de uma base de dados. É nos campos que as informações ficam armazenadas.

Para uma otimização da base de dados, antes de utilizar, devemos definir os campos que desejamos usar, e especificar o que cada um pode conter.

Campos de Banco de Dados

Tipos de campos no MySQL

O MySQL oferece campos bem comuns, que até mesmo um programador novato já deve ter visto.

Alguns deles estão aqui listados:

- CHAR
- VARCHAR
- INT
- FLOAT
- DATE
- TEXT/BLOB

Banco de Dados

Atividade 1

O que é MySQL?

Quais as vantagens de usar MySQL?

O que é Banco de Dados?

Qual o conceito de banco de dados?

O que é uma tabela em banco de dados?

O que é um registro em banco de dados?

O que é um campo em banco de dados?

O que são 'tipos de dados' em banco de dados?

Campos de Banco de Dados

CHAR (M)

Os campos CHAR são usados para caracteres alfanuméricos, como endereços e nomes. Seu tamanho é fixo e instaurado ao ser criado. Um campo do tipo CHAR pode ter de 1 a 255 caracteres.

Exemplo:

```
bairro CHAR(30);
```

Define um campo chamado 'bairro', que pode conter até trinta letras. Observe que não há acentos no nome do campo, pois muitos servidores não acentuam, e sua tabela teria difícil acesso.

Campos de Banco de Dados

VARCHAR(M)

Sua estrutura e funcionamento é idêntico ao campo anterior, salvo que no tipo CHAR, o tamanho definido é fixo, e mesmo que não usado, aquele espaço em disco é alocado. Já o campo VARCHAR, aloca apenas o espaço necessário para gravação. Contudo, vale lembrar que trocamos espaço por velocidade, pois este campo é 50% mais lento que o anterior.

Exemplo:

bairro VARCHAR(30);

Define um campo chamado bairro que pode conter até trinta letras. Se você preencher apenas duas, o campo não ocupará todos os trinta bytes, mas apenas dois.

Campos de Banco de Dados

CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT

			MIN	MAX
CHAR	String de tamanho fixo. Sempre é completada com espaços a direita até o tamanho definido		1	255 caracteres
		OBS	Espaços excessivos são removidos quando o valor é trazido. Os valores são ordenados e comparados ignorando caixas altas e baixas de acordo com a codificação padrão, a menos que seja fornecido uma chave binária.	
VARCHAR	String de tamanho variável		1	255 caracteres
		OBS	Os valores são ordenados e comparados ignorando caixas altas e baixas de acordo com a codificação padrão, a menos que seja fornecido uma chave binária. Nota: Espaços excessivos são removidos quando o valor é inserido.	
TINYTEXT			0	255 ($2^8 - 1$) caracteres
TEXT			0	65535 ($2^{16} - 1$) caracteres
MEDIUMTEXT			0	16777215 ($2^{24} - 1$) caracteres
LONGTEXT			0	4294967295 ($2^{32} - 1$) caracteres

Campos de Banco de Dados

INT(M) [Unsigned]

O campo INT, que como o próprio número diz, suporta o conjunto dos números inteiros, originalmente numa variação de -2147483648 a 2147483647 .

O parâmetro Unsigned pode ser passado, excluindo os números negativos, proporcionando um intervalo de 0 até 4294967295 .

Exemplo 01:

`temperatura INT;`

Campo com nome temperatura que pode ter valores positivos e negativos.

Exemplo 02:

`temperatura INT unsigned;`

Campo com nome temperatura que pode ter apenas valores positivos.

Campos de Banco de Dados

TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT

Type	Storage	Minimum Value	Maximum Value
	(Bytes)	(Signed/Unsigned)	Signed/Unsigned)
TINYINT	1	-128	127
		0	255
SMALLINT	2	-32768	32767
		0	65535
MEDIUMINT	3	-8388608	8388607
		0	16777215
INT	4	-2147483648	2147483647
		0	4294967295
BIGINT	8	-9223372036854775808	9223372036854775807
		0	18446744073709551615

Campos de Banco de Dados

FLOAT[(M,D)]

Os pontos flutuantes (FLOAT) representam pequenos números decimais, e são usados para representar números com maior precisão.

Exemplo:

```
voltagem_cadeira_eletrica FLOAT(4,2);
```

Float válido: 1324.50

A Linguagem SQL

- ▶ LDD - Linguagem de Definição de Dados (ou DDL)
- ▶ LMD - Linguagem de Manipulação de Dados (DML)

SQL -

Comandos

- ▶ **/* Comentário */** Comentário dentro ou antes de um comando SQL
- ▶ **Comment** Inclui no dicionário de dados um comentário relativo a uma tabela ou coluna
- ▶ **Commit** Efetiva no banco de dados (tornando permanentes) as modificações que já foram completadas
- ▶ **Grant** Atribui privilégios relativos a tabelas e visões
- ▶ **Revoke** Revoga privilégios relativos ao BD ou acesso a tabelas
- ▶ **Rollback** Desfaz, em caso de falha do sistema, qualquer alteração que ainda não tenha sido efetivada, de modo que a integridade do BD seja mantida. Permite ao usuário desfazer qualquer alteração antes de um COMMIT .

LDD - Linguagem de Definição de Dados (ou DDL)

- ▶ **Create Table** - cria Tabelas
- ▶ **Create View** - cria Visões do Banco de Dados
- ▶ **Create Index** - define índice sobre uma Tabela
- ▶ **Alter Table** - altera a estrutura de Tabelas existentes
- ▶ **Drop Table** - elimina uma Tabela do Banco de Dados
- ▶ **Drop View** - elimina uma Visão do sistema
- ▶ **Drop Index** - elimina um índice definido para uma Tabela

SQL – Criando Tabelas

► Criando as Tabelas

Formato: **Create Table** <nome-da-tabela>

(<descrição das colunas>);

(<descrição das chaves>);

<nome da tabela> - Nome da tabela a ser criada

<Descrição das colunas> - É uma lista de colunas (campos) e seus respectivos tipos de dados (smallint, char, varchar, integer, money, decimal, float, real, date, time, timestamp, logical)

<descrição das chaves> - É a lista de colunas que são tratadas como chave primária e chave estrangeira.

Tipos de dado (Domínio) em SQL

- **char(n).** String com tamanho fixo n .
- **varchar(n).** String com tamanho variavel, no máximo até n .
- **int.** Inteiro
- **smallint.** Inteiro pequeno
- **numeric(p,d).** Numero de pontos, precisão de p digitos, com n digitos a direita.
- **real, double precision.** Numero real e ponto flutuante com precisão da máquina.
- **float(n).** Ponto flutuante com n digitos de precisão

SQL – Criando Tabelas

Create Table Cliente

```
(código_cliente    smallint    not null,  
nome_cliente      char(20),  
endereço          char(30),  
cidade            char(15),  
CEP               char(08),  
UF                char(02),  
CGC               char(20),  
IE                char(20),
```

não pode ser nulo

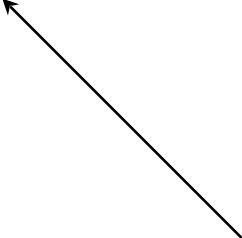
```
Primary Key (código_cliente)); /* Chave Primária */
```

SQL – Criando Tabelas

Create Table Vendedor

(código_vendedor	smallint	not null,
nome_vendedor	char(20),	
salario_fixo	money,	
faixa_comissão	char(01),	

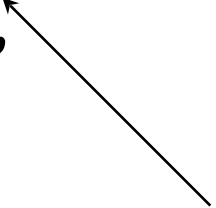
Primary Key (código_vendedor); /* Chave Primária */



Create Table Produto

(código_produto	smallint	not null,
unidade	char(03),	
descrição_produto	char(30),	
val_unit	money,	

Primary Key (código_produto); /* Chave Primária */



SQL – Criando Tabelas

Create Table Pedido

num_pedido	integer	not null,
prazo_entrega	smallint	not null,
código_cliente	smallint	not null,
código_vendedor	smallint	not null,

Primary Key (num_pedido),	/* Chave Primária */
Foreign key (código_cliente)	/* Chave Estrangeira */
references Cliente	
Foreign key (código_vendedor)	/* Chave Estrangeira */
references Vendedor);	

SQL – Criando Tabelas

Create Table Item_do_Pedido

```
(num_pedido      integer    not null,  
 código_produto  smallint   not null,  
 quantidade      decimal,  
 código_vendedor smallint   not null,
```

```
Primary Key (num_pedido,código_produto),      /* Chave Primária */  
Foreign key (num_pedido)                      /* Chave Estrangeira */  
           references Pedido  
Foreign key (código_produto)                  /* Chave Estrangeira */  
           references Produto);
```

Linguagem de Manipulação de Dados (ou DML)

- ▶ A DML da SQL apresenta os seguintes comandos para manipulação dos dados em um banco de dados:
- ▶ Comandos de modificação de dados:
 - ▶ Inclusão de uma ou mais linhas: ***Insert***
 - ▶ Atualização de uma ou mais linhas: ***Update***
 - ▶ Exclusão de uma ou mais linhas: ***Delete***
- ▶ Comando de consulta - ***Select***

SQL – Inserindo Registros

Adicionando tuplas (linhas) à Tabela:

Formato: INSERT INTO <nome da tabela>
(<nome das colunas>)
VALUES (<valores>);

Exercício: Adicionar o produto 'Queijo' à tabela produto

```
Insert into Produto  
values (25, 'Kg', 'Queijo', 0.97); /* valores posicionais  
com os respectivos  
atributos */
```

SQL – Modificando Registros

Modificando uma Tupla (linha) da Tabela

Formato: Update <nome da tabela>
 set <nome da coluna> = valor
 where <condição>;

Update Produto
 set val_unit = 1.62
 where descrição_produto = 'Chocolate';

Update Vendedor
 set salário_fixo = (salário_fixo * 1.27) + 100;

SQL – Excluindo Registros

Apagando tuplas (linhas) da Tabela

Formato: DELETE FROM <nome da tabela>
 where <condição>;

```
DELETE FROM Vendedor  
          where faixa_comissão IS NULL;
```

SQL – Selecionando Registros

Formato: Select <lista de atributos>
 From <lista de relações>;
 Where <lista de predicados>

Cláusula Select: Operação de Projeção da Álgebra. Contém a lista de atributos desejados no resultado da consulta.

Cláusula From: Operação de Produto Cartesiano da Álgebra. Contém a lista das relações que serão utilizadas na consulta.

Cláusula Where: Operação de Seleção da Álgebra. Contém a lista de predicados.

SQL – Selecionando Registros

Selecionando Colunas Específicas da Tabela:

Select descrição_produto, unidade, val_unit *from* Produto;

Select nome_cliente, endereço, CGC *from* Cliente;

Selecionando todas as Colunas da Tabela:

Formato: *Select* * *from* <tabela>;

Select * *from* Vendedor;

SQL – Selecionando Registros

Formato: Select <nome das colunas> from <tabela>
Where <restrições>;

Comparações na Cláusula WHERE:

Where <nome da coluna> <operador> <valor>

Observações:

- Quando a coluna é do tipo caractere, o <valor> deve estar entre aspas ('). Ex.: 'PARAFUSO'.

Observações – Comando Select:

- Na cláusula *Select* devem ser especificadas as colunas (ou campos) que deverão aparecer no resultado da consulta.
- Caso se deseje todos os campos da tabela no resultado de uma consulta, então um * pode ser utilizado após a palavra-chave *Select*.
- O resultado da execução de uma ou consulta (ou query) é uma tabela “virtual” (não básica - ou seja, não original no banco de dados).

SQL – Operadores Relacionais

=	Igual
< >	Diferente
<	Menor do que
>	Maior do que
<=	Menor ou igual do que
>=	Maior ou igual do que

```
Select num_pedido, código_produto, quantidade  
from item_do_pedido  
where quantidade = 35;
```

```
Select nome_cliente from cliente  
where cidade = 'Niterói';
```

SQL – Operadores Lógicos

AND “e” lógico
OR “ou” lógico”
NOT negação

```
Select descrição_produto    from produto  
      where unidade = 'M' AND val_unit = 1.05;
```

```
Select nome_cliente, endereço  
      from cliente  
      where (CEP >= '30077000' AND CEP <=  
          '30079000')  
          OR cidade = 'São Paulo';
```

```
Select num_pedido        from pedido  
      where NOT (prazo_entrega = 15);
```

SQL – Operador Between

Where <nome coluna> **BETWEEN** <valor1> AND <valor2>

Where <nome coluna> **NOT BETWEEN** <valor1> AND <valor2>

```
Select código_produto, descrição_produto  
      from produto  
      where val_unit between 0.32 and 2.00;
```

Observação: Inclusive os limites inferior e superior.

SQL – Operador Is NULL

Where <nome da coluna> IS NULL

Where <nome da coluna> IS NOT NULL

```
Select *  
    from cliente  
    where IE IS NULL
```

SQL - Ordenando Dados Seleccionados

- ▶ Quando se realiza uma seleção, os dados recuperados não estão ordenados.
- ▶ A SQL prevê a cláusula ORDER BY para realizar a ordenação dos dados selecionados.

Formato: Select <nome das colunas>

from <tabela>

Where <condições>

Order by <nome das colunas>

OU:

Order by <número das colunas>

ASC ou DESC

SQL - Ordenando Dados Seleccionados

- ▶ A informação <número das colunas> se refere à posição relativa das colunas quando for apresentado o resultado da consulta, e não à posição na tabela original, contada da esquerda para a direita.
- ▶ As palavras **ASC** e **DESC** significam respectivamente, *ascendente* e *descendente*. A forma ascendente é assumida como padrão (default).

SQL - Ordenando Dados Seleccionados

```
Select nome_vendedor, salario_fixo  
from vendedor  
order by nome_vendedor;
```

```
Select nome_cliente, cidade, UF  
from cliente  
order by UF DESC, cidade DESC;
```

```
Select descrição_produto, val_unit  
from produto  
Where unidade = 'M'  
order by 2 ASC;
```

Funções Embutidas da SQL

- ► Também chamadas funções de agregação ou de grupo.
- ► São elas:
 - COUNT – conta o número de registros ou número de valores em uma coluna
 - SUM – somatório de valores de um campo numérico
 - MAX – valor máximo em uma determinada coluna
 - MIN – valor mínimo em uma determinada coluna
 - AVG – média dos valores de um campo numérico

SQL - Utilizando Funções

► *Buscando Máximos e Mínimos (MAX, MIN)*

```
Select  MIN (salário_fixo), MAX (salário_fixo)
        from vendedor;
```

► *Totalizando Colunas (SUM)*

```
Select  SUM (quantidade)
        from item_do_pedido
        Where código_produto = '78';
```

SQL - Utilizando Funções

► *Calculando Médias (AVG)*

```
Select  AVG (salário_fixo)
        from vendedor;
```

► *Contando as Linhas (COUNT)*

```
Select  COUNT(*)
        from vendedor
        Where salário_fixo > 2500;
```

SQL - Utilizando Funções:

COUNT

- ▶ *Única função que usa * como parâmetro;*
- ▶ *Conta o número de linhas na tabela (cardinalidade);*

Seja: empregado <matric, nome, depto, salario>
dependente <matric_E, nome_dep>

```
Select  COUNT (distinct Salario)
        from empregado; /* número de diferentes salarios*/
```

```
Select  nome
        from empregado
        Where (Select COUNT(*)
                from dependente
                Where matric = matric_E) >= 2; /*mais de 2 dependentes*/
```

SQL - Utilizando Cláusula Distinct

- *Problema:* Quais as unidades de produtos, diferentes, existem na tabela produto ?

```
Select  DISTINCT unidade,  
        from produto;
```

- A tabela Produto pode possuir vários produtos com a mesma unidade de medida (L, M, Kg ...). A SQL não retira automaticamente estas repetições. Para sabermos quais unidades existem, sem repetições, usamos a cláusula **DISTINCT**.

SQL - Recuperando Dados de Várias Tabelas

- ▶ *Problema:* Juntar a tabela CLIENTE com PEDIDO.

```
Select  nome_cliente, pedido.código_cliente, num_pedido  
        from cliente, pedido;
```

- ▶ Como podemos ver, o **qualificador de nome** consiste no nome da tabela seguido de um ponto e o nome da coluna na tabela, por exemplo:
 - ▶ Coluna DESCRIÇÃO da tabela PRODUTO
PRODUTO.descrição_produto
- ▶ Os qualificadores de nomes são utilizados para efetivar o JOIN

SQL - Recuperando Dados de Várias Tabelas

- *Problema:* Que clientes fizeram os pedidos ?

```
Select  nome_cliente, pedido.código_cliente, num_pedido
        from cliente, pedido
        Where cliente.código_cliente = pedido.código_cliente;
```

- Podemos utilizar as cláusulas LIKE, NOT LIKE, IN, NOT IN, NULL, NOT NULL e misturá-las com os operadores AND, OR e NOT, dentro de uma cláusula WHERE na junção entre tabelas.

SQL - Recuperando Dados de Várias Tabelas

- *Problema:* Quais clientes que têm prazo de entrega superior a 15 dias e que pertencem aos estados de São Paulo (SP) ou Rio de Janeiro (RJ) ?

```
Select  nome_cliente, UF, prazo_entrega
        from cliente, pedido
        Where UF IN ( 'SP' , 'RJ' ) AND
                prazo_entrega > 15 AND
                cliente.cod_cliente = pedido.cod_cliente;
```

SQL - Recuperando Dados de Várias Tabelas

- **Problema:** Mostrar os clientes e seus respectivos prazos de entrega, ordenados do maior para o menor.

```
Select  nome_cliente, prazo_entrega  
        from cliente, pedido
```

```
Where  cliente.código_cliente = pedido.código_cliente
```

```
Order by prazo_entrega desc;
```



Chamada de
Equação de Junção

SQL - Tipos de Junção

Seja: **empregado** <matric, noem depto, salario, ger>
departamento <numd, nomed>

- ▶ Padrão: INNER JOIN (junção interna)
 - ▶ Cláusula de junção no where

Ex1.: Empregados e seus gerentes

```
Select E.nome As nome_emp, G.nome As nome_ger  
from Empregado E, Empregado G  
Where E.ger = G.matric
```

SQL - Tipos de Junção

► NATURAL JOIN

- Nenhum predicado de junção é especificado;
- Uma condição implícita é criada, com os atributos de mesmo nome.

Ex2.: Nome e salário dos empregados do departamento de informática

```
Select nome, salario
```

```
from Empregado JOIN Departamento ON depto = numd  
Where nomed = 'informatica'
```

```
Select nome, salario
```

```
from Empregado NATURAL JOIN
```

```
(Departamento as D (nomed, depto))
```

```
Where nomed = 'informatica'
```

SQL - Outros

Exemplos

- ► Combinar Resultados de Pesquisas (UNION):
 - ► **Problema:** Listar os nomes e códigos dos vendedores que têm salário fixo > R\$ 1.000,00 e clientes que residem no Rio.
 - Select código = código_cliente, nome = nome_cliente from cliente
 - where UF = 'RJ'
 - UNION
 - Select código_vendedor, nome_vendedor
 - from vendedor
 - where salário_fixo > 1000;

SQL - Outros

Exemplos

► Fazer um JOIN da tabela com ela mesma:

► **Problema:** Para todos os vendedores, listar os nomes e salários dos vendedores com salários fixo menores que os de outros vendedores .

```
Select nome_vendedor = V1.nome_vendedor,  
       salario_menor = V1.salario_fixo,  
       nome_maior = V2.nome_vendedor,  
       salario_maior = V2.salario_fixo,  
from Vendedor V1, Vendedor V2  
Where V1.salario_fixo < V2.salario_fixo  
order by 1;
```

Exercício

S

- ▶ Selecione todos os campos da tabela projeto
- ▶ Selecione o código do projeto e as horas de cada projeto
- ▶ Liste o código de todos os empregados alocados em um projeto

EMP

EMPID	ENOME
E101	Sílvio
E105	João
E110	Adalto
E115	Sandra

PROJETO

PROJ#	PROJNOME	ORÇAMENTO
P10	HUDSON	500000
P15	COLUMBIA	350000
P20	PROSPECTUS	350000
P23	ANACONDA	600000

ALOCAÇÃO

EMPID	PROJ#	HORAS
E101	P10	200
E101	P15	300
E105	P10	400
E110	P10	250
E110	P20	350
E110	P15	700
E115	P10	300

Exercícios

- ▶ Liste o nome dos projetos com orçamento maior que 400000
- ▶ Liste os códigos dos empregados que trabalham no projeto P10
- ▶ Liste os códigos dos empregados que trabalham no projeto P10 com mais de 200 Horas
- ▶ Qual o nome do funcionário E105

EMP	
EMPID	ENOME
E101	Sílvia
E105	João
E110	Adalto
E115	Sandra

PROJETO		
PROJ#	PROJNOME	ORÇAMENTO
P10	HUDSON	500000
P15	COLUMBIA	350000
P20	PROSPECTUS	350000
P23	ANACONDA	600000

ALOCAÇÃO		
EMPID	PROJ#	HORAS
E101	P10	200
E101	P15	300
E105	P10	400
E110	P10	250
E110	P20	350
E110	P15	700
E115	P10	300